

Name: Fatima Khurram

Sap I'd : 56652

Section: BSCS-5B

Game Programming

Assignment:

Multiplayer Game Code Architecture Design

Project: Last Stand (3D Tactical Team Shooter)

1. Game Overview:

Last Stand is a 3D multiplayer team shooter game (4v4) made for mobile platforms.

Core Gameplay Mechanics:

- **Buy Phase:** At the start of each round, players get 30 seconds to buy guns and grenades using in-game coins.
- **Main Objective:** Attackers try to plant a bomb at a specific site. Defenders must stop them or defuse the bomb. Each round lasts 2 minutes.
- **Hero Abilities:** Players choose characters with special powers, like a speed dash, a scanning drone, or an energy shield.

2. High-Level Architecture:

This game uses a **Server-Authoritative Model**. This means the main online server makes all the important decisions to stop players from cheating. The mobile phone (Client) only sends touch inputs to the server, and the server tells the phone what visuals to show.

Major Systems Explained:

1. **Matchmaking System:** Finds 8 players online with similar skill levels and connects them together into a new game session.
2. **Player System:** Controls the 3D character movement, jumping, animations, and touch controls on the mobile screen.
3. **Combat System:** Checks when a player fires a gun, calculates if the bullet hit the enemy, and reduces the enemy health.
4. **Inventory & Economy System:** Tracks how many coins a player has, handles the gun shop menu, and counts the bullets left in the gun.
5. **Game State Management:** Controls the flow of the match (e.g., changing the game from the "Buy Phase" to the "Action Phase" and deciding who wins).
6. **UI System:** Shows all the menus on the screen, like the health bar, mini-map, weapon shop, and final scorecards.

3. Code Structure:

We organize the code using **Classes and Components** so that different parts of the game do not mix up and create bugs.

Core Classes & Their Responsibilities:

A. Global Managers

- **NetworkManager:** Connects the mobile app to the internet server and handles data sharing.
- **GameManager:** Controls the main rules, timers, and match rounds.
- **EconomyManager:** Calculates how many coins to give a player when they kill an enemy or win a round.

B. Component Scripts (Attached to Players & Objects)

- **PlayerController:** Takes touch inputs from the mobile screen joystick and moves the 3D character.
- **Health:** Keeps track of the player's life points (HP). It triggers the death screen when HP drops to zero.
- **WeaponSystem:** Handles gun functions like shooting, reloading, and playing gunshot sounds.

How Systems Communicate (The Observer Pattern)

Instead of linking all scripts directly, the game uses an **Event System**. Systems just "listen" for events.

- **Example Flow (When a player kills an enemy):**
 1. WeaponSystem detects a hit → Tells the enemy's Health script to take damage.
 2. Health script reaches 0 → It shouts out a message: *"Player X is Dead!"*
 3. GameManager hears this message → Reduces the team count.
 4. EconomyManager hears this message → Gives +200 coins to the killer.
 5. UI System hears this message → Shows the kill icon on the top corner of the screen.

4. Design Decisions (Technical Reasoning)

- **Why use a Server-Authoritative system?**

If we calculate damage and movement on the mobile device, hackers can easily change their game files to get infinite health or speed hacks. By making the server the "boss," cheating becomes impossible.

- **Why use the Observer Pattern for communication?**

It keeps the code clean. If we want to change or redesign the UI screens in the future, we can do it easily without touching or breaking the main shooting and movement code.

- **Why use Raycasting for shooting?**

Instead of creating physical 3D bullets flying through the air, Raycasting shoots an instant invisible straight line. This makes the shooting feel fast, crisp, and smooth even on low-end mobile phones.